Farmers' Market Vendor Cooperative Management System

By: Kenneth Keopraseuth, Aishwarya Ramkumar, Ruqia Abdiwahab, Ilya Shkurko, John Xiu

Introduction

Project Goal

- Build a database to manage vendor registrations, permits, stall bookings, inspections, and sales.

Key Features

- Automate business rules and track compliance
- Monitor vendor performance with KPIs
- Generate reports to support market operations

Purpose

- Streamline daily tasks and improve decision-making



Business Description

- Farmers' market connects vendors with the community through local and handmade goods
- Managing vendors, permits, stalls, and inspections is currently manual and difficult
- Database will improve organization, accuracy, and efficiency

Tactical Goals

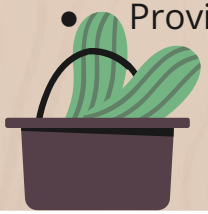
- Implement database-driven market management within 1–3 years
- Provide reports to improve planning and optimize stall usage
- Automate permit and inspection checks for vendor compliance

Strategic Goals

- Make vendor management processes more efficient.
- Help vendors book stalls quickly and avoid double-booking.
- Use data on sales, attendance, and inspections to guide future improvements.

Operational Goals

- Upgrade stall booking process with automated systems within 3 months to reduce no-show rates.
- Improve vendor spaces to provide more access to stalls.
- Provide monthly education to vendors on compliance for permits and inspections.



ER Diagram

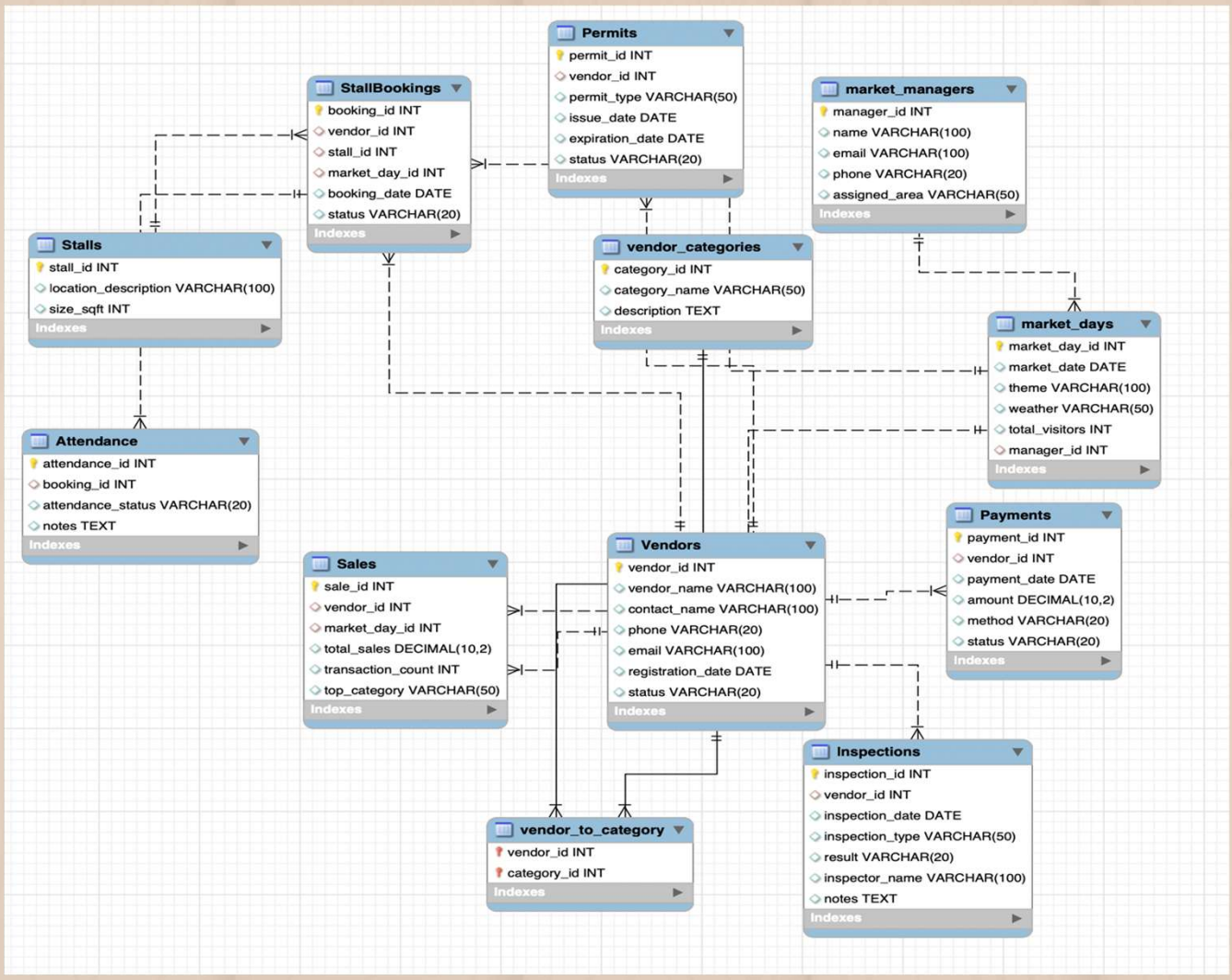

Permits
permit_id INT
vendor_id INT
permit_type VARCHAR(50)
issue_date DATE
expiration_date DATE
status VARCHAR(20)
Indexes
StallBookings
booking_id INT
vendor_id INT
stall_id INT
market_day_id INT
booking_date DATE
status VARCHAR(20)
Indexes
market_managers
manager_id INT
name VARCHAR(100)
email VARCHAR(100)
phone VARCHAR(20)
assigned_area VARCHAR(50)
Indexes
Stalls
stall_id INT
location_description VARCHAR(100)
size_sqft INT
Indexes
vendor_categories
category_id INT
category_name VARCHAR(50)
description TEXT
Indexes
market_days
market_day_id INT
market_date DATE
theme VARCHAR(100)
weather VARCHAR(50)
total_visitors INT
manager_id INT
Indexes
Attendance
attendance_id INT
booking_id INT
attendance_status VARCHAR(20)
notes TEXT
Indexes
Payments
payment_id INT
vendor_id INT
payment_date DATE
amount DECIMAL(10,2)
method VARCHAR(20)
status VARCHAR(20)
Indexes
Sales
sale_id INT
vendor_id INT
market_day_id INT
total_sales DECIMAL(10,2)
transaction_count INT
top_category VARCHAR(50)
Indexes
Vendors
vendor_id INT
vendor_name VARCHAR(100)
contact_name VARCHAR(100)
phone VARCHAR(20)
email VARCHAR(100)
registration_date DATE
status VARCHAR(20)
Indexes
Inspections
inspection_id INT
vendor_id INT
inspection_date DATE
inspection_type VARCHAR(50)
result VARCHAR(20)
inspector_name VARCHAR(100)
notes TEXT
Indexes
vendor_to_category
vendor_id INT
category_id INT
Indexes

Table Structures Pt.1

Vendors

- Stores each vendor's basic information like name, contact, business type, and status.

Permits

- Tracks permits issued to vendors, including type, issue/expiration dates, and validity.

Stalls

- Holds details about stall locations and their sizes at the market.

StallBookings

- Records which vendor booked which stall on which day, including booking status.

Inspections

- Logs health and safety inspection results for vendors, along with inspector notes.

Sales

- Tracks how much each vendor earned, how many sales they made, and their top-selling category.

Table Structures Pt.2

market_days

- Market events (date, theme, weather, visitors)

vendor_categories

- Types of vendor businesses (produce, crafts, flowers, etc.)

vendor_to_category

- Mapping of vendors to their business categories (junction table linking each vendor_id to a category_id)

payments

- Vendor payments (amount, method, status)
- Linked to vendors

attendance

- Vendor attendance (present, no show, cancelled)
- Linked to vendors and market days

market_managers

- Market staff (name, contact, assigned area)

Normalization Process

Normal forms applied:

- First normal form :
  - All tables have atomic values (no repeating groups)
  - Example: The products table stores one product per row with no nested lists.
- Second Normal Form:
  - All non-key attributes are fully functionally dependent on the primary key.
  - Example: Sales table links total sales and transactions to a single vendor_id.
- Third Normal Form:
  - All fields depend only on the main ID, not on other fields.
  - Example: VendorCategories stores category descriptions separately from the Vendors table.

SQL Commands for Table Creation

- Created 12 tables using create table commands
- Defined primary keys and foreign keys to link tables
- Inserted sample data using insert into commands

```sql
CREATE TABLE Sales (
  sale_id           INT           PRIMARY KEY,
  vendor_id         INT,
  market_day_id     INT,
  total_sales       DECIMAL(10,2),
  transaction_count INT,
  top_category      VARCHAR(50),
  FOREIGN KEY (vendor_id)     REFERENCES Vendors(vendor_id),
  FOREIGN KEY (market_day_id) REFERENCES market_days(market_day_id)
);
```

Views/KPIs

The database uses SQL views and queries to track and evaluate the performance of key metrics. Data from tables such as Vendors, Sales, Market_Days, Stall_Bookings, Attendance, and Inspections are utilized for accurate reports from each Key Performance Indicator.

Top Vendors By Sales

- Vendor 10 business type: eggs with total sales of $950.
- Strong performance above average

Vendor No-Show Rate

- Vendor no-show rate is 33% out of a total of 10 bookings.
- This is only a small %, however there is a possibility for a loss of revenue due to absences.

Average Sales Per Market Day

- High overall performance with an average of 725$ sales

Inspection Compliance Rate

- Inspection compliance rate is at 70%. Demonstrating strong adherence to regulations, while there is also room for improvement.

Triggers

Trigger Name	Purpose	Table
trg_prevent_expired_permit_booking	Prevent stall booking with expired permit	StallBookings
trg_default_sales_category	Set a default category if none is provided	Sales
trg_log_vendor_status_change	Log status changes of vendors into audit table	Vendors
trg_auto_notes_no_show	Auto-fill notes if attendance is marked no_show and left empty	Attendance

SQL code for each trigger is included in the project SQL submission file.

Stored Procedure

Since our project is focused on vendors, stalls, sales, inspections, and KPIs, the adequate stored procedure would be one that adds a new vendor, books a stall, and inserts an initial inspection.

- sp_RegisterVendorAndBookStall
    - This stored procedure simplifies onboarding a new vendor, and it would insert the vendor's details into the Vendor table, book a stall for the vendor in the StallBooking table, then schedule an initial inspection in the Inspection table

Stored Procedure is very helpful in our project because it automates vendor onboarding, which reduces errors in manual entry. It also improves compliance by making sure that inspections are scheduled from the start.

Stored Procedure Script

```
CREATE PROCEDURE sp_RegisterVendorAndBookStall(
  IN p_vendor_name       VARCHAR(100),
  IN p_contact_name      VARCHAR(100),
  IN p_phone             VARCHAR(20),
  IN p_email             VARCHAR(100),
  IN p_category_id       INT,
  IN p_registration_date DATE,
  IN p_status            VARCHAR(20),
  IN p_stall_id          INT,
  IN p_market_day_id     INT,
  IN p_inspector_name    VARCHAR(100)
)
BEGIN
  DECLARE v_vendor_id INT;

  -- 1) Add the new vendor
  INSERT INTO Vendors
    (vendor_name, contact_name, phone, email, registration_date, status)
  VALUES
    (p_vendor_name, p_contact_name, p_phone, p_email, p_registration_date, p_status);

  SET v_vendor_id = LAST_INSERT_ID();

  -- 2) Link to its category
  INSERT INTO vendor_to_category(vendor_id, category_id)
  VALUES (v_vendor_id, p_category_id);

  -- 3) Book a stall
  INSERT INTO StallBookings
    (vendor_id, stall_id, market_day_id, booking_date, status)
  VALUES
    (v_vendor_id, p_stall_id, p_market_day_id, CURDATE(), 'Confirmed');

  -- 4) Schedule an initial inspection
  INSERT INTO Inspections
    (vendor_id, inspection_date, inspection_type, result, inspector_name, notes)
  VALUES
    (v_vendor_id, CURDATE(), 'Initial', 'Pending', p_inspector_name,
     'Initial inspection scheduled');

END$$
```

Sample Data

Vendors Table (Sample)

vendor_id	vendor_name	status
1	Vendor 1	Active
10	Vendor 10	Suspended

Permits Table (Sample)

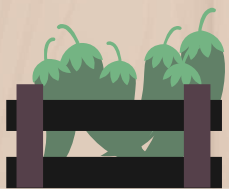
permit_id	vendor_id	expiration_date	status
9	9	2024-09-01	Expired
10	10	2024-10-01	Expired

Attendance Table (Sample)

attendance_id	booking_id	attendance_status	notes
1	1	present	On time
3	3	no_show	Did not arrive

Sales Table (Sample)

sale_id	vendor_id	total_sales	top_category
1	1	500.00	Produce
10	10	950.00	Eggs



Queries and Reports

An example of running queries would be a script like this:

```
SELECT
   v.vendor_name,
   SUM(s.total_sales) AS total_sales_amount
FROM
   Vendors v
JOIN
   Sales s ON v.vendor_id = s.vendor_id
GROUP BY
   v.vendor_name
ORDER BY
   total_sales_amount DESC
LIMIT 5;
```

The output would be something like this:

Vendor Name	Total Sales Amount
Sunny Farms	950
Fresh Blooms	825
Green Goods	700
Sweet Treats	680
Organic Roots	650



An example of average sales per market day script is:

```
SELECT
   md.market_date,
   AVG(s.total_sales) AS avg_sales_per_day
FROM
   Market_Days md
JOIN
   StallBookings sb ON md.market_day_id =
sb.market_day_id
JOIN
   Sales s ON sb.vendor_id = s.vendor_id
GROUP BY
   md.market_date
ORDER BY
   md.market_date;
```

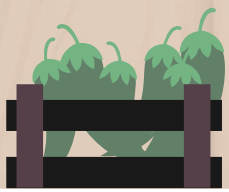
Market Date	Average Sales Per Day
04/01/025	750
4/8/2025	720
4/15/2025	700

There are multiple reports we could generate from the data:

- Top Vendor Report
  - Identifies highest-performing vendors by sales
- Sales Performance Over Time Report
  - Monitors market health and profitability
- Inspection Compliance Report
  - Ensure public health and safety standards are met

Conclusion

- Our Farmers' Market Vendor Cooperative Management System utilizes a SQL database to manage vendor registrations, permits, stall bookings, inspections, attendance, and sales.
- **Key Tools:**
  - 12 core tables for structured data.
  - 4 Views/KPIs monitor performance.
  - Triggers automate tasks.
  - Stored procedures simplifies vendor registration and stall bookings.



References

Mickelson, B. (2023). *Improving the Use of Key Performance Indicators (KPIs) in Operations Management*.

Coronel, C., & Morris, S. (2023). Database systems: Design, implementation, & management (14th ed.). Cengage.

